Healthcare Platform

Full Stack Technical Reference Document

Navya Akurathi

Senior Java Full Stack Developer | 10+ Years Experience

United States

Stack: Java 17 · Spring Boot 3 · Kafka · GCP · Angular 17 · React 18

GitHub: github.com/nakurathi/Dev

Table of Contents

1. Project Overview

The Healthcare Platform is a production-grade, cloud-native microservices application built to demonstrate senior-level full stack engineering across the complete technology stack. The system manages patients, appointments, billing, and audit trails using modern patterns including event-driven architecture, sealed domain events, circuit breakers, and role-based security.

Repository
https://github.com/nakurathi/Dev

1.1 Architecture Overview

The platform follows a layered, event-driven architecture:

- REST API Layer — Spring Boot 3 controllers serve JSON over HTTPS with JWT authentication
- Domain Layer — Java 17 sealed interfaces and records model all business events
- Kafka Event Bus — Domain events flow between services via GCP-hosted Kafka topics
- GCP Services — Spanner (database), Pub/Sub (messaging), Secret Manager (secrets), GCS (files)
- Frontend — Angular 17 (standalone components, Signals) and React 18 (Vite, Redux Toolkit)
- Infrastructure — Docker + Kubernetes on GKE, GitHub Actions CI/CD pipeline

1.2 Technology Stack

Layer	Technology	Purpose
Backend	Java 17, Spring Boot 3.2	REST APIs, business logic, GCP integration
Messaging	Apache Kafka	Async domain event streaming between services
Cloud DB	GCP Cloud Spanner	Horizontally scalable globally distributed SQL
Messaging	GCP Pub/Sub	Managed push/pull messaging and notifications
Secrets	GCP Secret Manager	Centralized secret storage, sm:// Spring injection
Storage	GCP Cloud Storage	File uploads, audit NDJSON logs, reports
Frontend	Angular 17	Standalone components, Signals, Material UI
Frontend	React 18 + Vite	Redux Toolkit, React Query, Tailwind CSS
Security	Spring Security 6 + JWT	Stateless OAuth2 resource server, role-based access
Resilience	Resilience4j	Circuit breakers, retries, fallbacks for GCP calls
Observability	Micrometer + Prometheus	Custom metrics, timers, health indicators
Container	Docker + GKE	Multi-stage builds, Kubernetes rolling deployments
CI/CD	GitHub Actions	Build → test → Docker push → GKE deploy

Layer	Technology	Purpose
		pipeline

2. Backend — Spring Boot 3 & Java 17

2.1 Application Entry Point

GcpApplication.java

The main Spring Boot entry point. Uses @SpringBootApplication which enables auto-configuration, component scanning, and bean registration across all packages under com.navya.gcp.

2.2 Domain Layer

DomainEvent.java — Sealed Interface + Java 17 Records

Java 17 Feature

Uses sealed interfaces and records — compile-time exhaustive pattern matching with no boilerplate

A sealed interface hierarchy models all business events flowing through Kafka. Each event is an immutable Java 17 record with automatic equals/hashCode/toString and no Lombok needed.

- AppointmentScheduled — fires when an appointment is booked
- AppointmentCancelled — fires when an appointment is cancelled with reason
- PatientRegistered — fires when a new patient is created
- InvoiceGenerated — fires when a billing invoice is created
- PaymentReceived — fires when a payment is processed

The sealed switch expression in DomainEventProducer routes each event to its Kafka topic. Because the interface is sealed, the compiler enforces that every case is handled — no forgotten event types:

```
String topic = switch (event) {
    case DomainEvent.AppointmentScheduled e -> "healthcare.appointments.scheduled";
    case DomainEvent.AppointmentCancelled e -> "healthcare.appointments.cancelled";
    case DomainEvent.PatientRegistered    e -> "healthcare.patients.registered";
    case DomainEvent.InvoiceGenerated     e -> "healthcare.billing.invoices";
    case DomainEvent.PaymentReceived      e -> "healthcare.billing.payments";
};
```

Dtos.java — Java 17 Records as DTOs

All request/response shapes are Java 17 records. Records are immutable, require zero boilerplate, and work with Bean Validation annotations on constructor parameters:

- AppointmentRequest — patientId, doctorId, scheduledAt (future validation), notes
- AppointmentResponse — the full appointment data returned to clients
- AppointmentStatusUpdate — status (enum-validated), reason
- InvoiceRequest — patientId, appointmentId, amount (min 0.01), dueDate
- InvoiceResponse — full invoice with status and payment timestamps
- PaymentRequest — invoiceId, amountPaid
- ApiResponse<T> — generic wrapper with success, message, data fields

Appointment.java / Billing.java — Cloud Spanner Entities

JPA-style entity classes annotated with Spring Cloud GCP Spanner annotations. @Table maps to the Spanner table, @PrimaryKey defines the row key, @Column maps fields to columns. Both entities use Instant for timestamps (Spanner TIMESTAMP type).

2.3 REST API Layer

Method	Endpoint	Auth Role	Description
POST	/api/v1/appointments	CLINICIAN, ADMIN	Schedule a new appointment, publishes Kafka event
GET	/api/v1/appointments/{id}	Any	Get appointment by ID
GET	/api/v1/appointments?patientId	Any	List appointments for a patient
PATCH	/api/v1/appointments/{id}/status	CLINICIAN, ADMIN	Update status, publishes cancel event if CANCELLED
DELETE	/api/v1/appointments/{id}	CLINICIAN, ADMIN	Cancel appointment
POST	/api/v1/billing/invoices	BILLING, ADMIN	Create invoice, publishes InvoiceGenerated event
GET	/api/v1/billing/invoices/{id}	BILLING, ADMIN	Get invoice by ID
POST	/api/v1/billing/payments	BILLING, ADMIN	Process payment, publishes PaymentReceived event
POST	/api/patients	Any	Create patient record in Spanner
GET	/api/patients/{id}	Any	Get patient by ID
POST	/api/pubsub/publish/orders	ADMIN	Publish to Pub/Sub orders topic
POST	/api/storage/upload	ADMIN	Upload file to GCS bucket
GET	/api/storage/signed-url/{name}	ADMIN	Generate V4 signed URL (default 15 min expiry)

AppointmentService.java — Business Logic

The service layer coordinates persistence and event publishing in a single @Transactional method. After saving to Spanner, it immediately publishes the corresponding DomainEvent to Kafka. This implements an outbox-lite pattern relying on Kafka's idempotent producer for at-least-once delivery guarantees.

GlobalExceptionHandler.java — RFC 7807 ProblemDetail

A @RestControllerAdvice that catches domain exceptions and maps them to RFC 7807 ProblemDetail responses. Spring Boot 3 natively supports ProblemDetail — no extra library needed. Returns structured JSON with type URI, status, detail, and instance fields instead of generic 500 errors.

2.4 REST Filters

CorrelationIdFilter.java

A Servlet filter (Order 1) that runs on every HTTP request. Reads the X-Correlation-Id header if present, or generates a new UUID. Injects it into the MDC (Mapped Diagnostic Context) so every log line for this request automatically includes the correlation ID. Also writes it back to the response header so clients can trace requests end-to-end across services.

RequestLoggingFilter.java

A Servlet filter (Order 2) that logs each request method, URI, response status, and elapsed milliseconds. Skips /actuator/health and /actuator/prometheus endpoints to avoid log noise from Kubernetes health probes.

3. GCP Integrations

3.1 GCP Pub/Sub

PubSubPublisherService.java

Wraps Spring Cloud GCP's PubSubTemplate for publishing messages to topics. Returns CompletableFuture<String> containing the message ID assigned by GCP. Supports publishing with custom attributes (key-value metadata) attached to the message — used for routing, filtering, and alerting by downstream subscribers.

PubSubSubscriberConfig.java

Spring @Configuration class that registers push-based message listeners at startup using PubSubTemplate.subscribe(). Two subscriptions are configured: orders-subscription and notifications-subscription. Each listener explicitly ACKs on success or NACKs on exception. NACKed messages are redelivered by GCP based on the subscription's ack deadline settings — providing automatic retry without any additional code.

OrderMessageHandler.java

Processes order payloads received from the orders Pub/Sub subscription. ACKs on successful processing, NACKs on any exception so the message is redelivered. Placeholder for deserializing JSON payload to domain objects and persisting to Spanner.

3.2 Cloud Spanner

Patient.java — Spanner Entity

Annotated with @Table and @PrimaryKey from Spring Cloud GCP Data Spanner. The patientId is a UUID string primary key. All timestamp fields use java.time.Instant which Spanner maps to its TIMESTAMP type. The entity supports statuses ACTIVE, INACTIVE, and ARCHIVED.

DDL to create the patients table in Spanner:

```
CREATE TABLE patients (
  patient_id   STRING(36)  NOT NULL,
  first_name   STRING(100) NOT NULL,
  last_name    STRING(100) NOT NULL,
  email        STRING(200),
  date_of_birth DATE,
  status       STRING(20)  NOT NULL
) PRIMARY KEY (patient_id);
```

PatientService.java

Provides standard CRUD operations via Spring Data SpannerRepository plus raw SQL execution via SpannerTemplate for analytics queries (e.g., COUNT by status). All write operations are annotated @Transactional. The countByStatus method demonstrates how to use parameterized Spanner SQL with named bind variables to avoid injection vulnerabilities.

3.3 GCP Secret Manager

SecretManagerService.java

Wraps the Secret Manager SDK client for programmatic CRUD on secrets. Supports fetching the latest version by name, creating a new secret with automatic replication, adding new versions to an existing secret, and version management. All SDK clients are opened in try-with-resources blocks to ensure proper connection cleanup.

SecretManagerConfig.java

Demonstrates the Spring Cloud GCP sm:// property source pattern. When spring.config.import: sm:// is set, Spring resolves properties like ${sm://projects/MY_PROJECT/secrets/db-password} at startup by fetching them from Secret Manager. This eliminates hardcoded secrets in configuration files and ensures secrets are never stored in Git.

3.4 Cloud Storage (GCS)

GcsStorageService.java

Full-featured GCS service covering the complete file lifecycle:

- upload(MultipartFile, objectName) — streams a Spring multipart upload directly to GCS using Storage.createFrom() for memory-efficient large file handling
- uploadBytes(byte[], objectName, contentType) — uploads raw byte arrays with explicit content type
- download(objectName) — returns file contents as byte array, empty array if not found
- stream(objectName) — returns InputStream for streaming large files without loading into memory
- generateSignedUrl(objectName, expiryMinutes) — generates a V4 signed URL for temporary client-side access without exposing credentials. Maximum 7 days (10080 minutes)
- listObjects(prefix) — lists all objects optionally filtered by a folder prefix
- delete(objectName) — returns boolean indicating whether the object existed and was deleted

ResilientGcsService.java

Wraps GcsStorageService with Resilience4j decorators. Each method is decorated with both a circuit breaker and retry policy, plus a fallback lambda that returns safe defaults (empty array, empty list) instead of propagating errors to callers. Uses the functional Decorators builder — no AOP proxy, fully explicit and unit-testable.

4. Apache Kafka Integration

4.1 Producer Configuration (KafkaProducerConfig.java)

Tuned for GCP-hosted Kafka (Cloud Managed Kafka or Confluent) with exactly-once semantics:

Setting	Value	Reason
enable.idempotence	true	Exactly-once delivery — duplicate suppression at broker
acks	all	All in-sync replicas must confirm — no data loss
retries	MAX_INT	Infinite retries within delivery.timeout.ms
max.in.flight.requests	5	Balanced throughput with ordering guarantees
compression.type	snappy	Reduces network bandwidth by ~50-70% on JSON payloads
batch.size	32 KB	Groups records together to reduce broker round trips
linger.ms	5	Waits 5ms to fill batch without sacrificing latency
delivery.timeout.ms	120,000	2 minute total delivery window before giving up

4.2 Consumer Configuration (KafkaConsumerConfig.java)

Manual offset commits ensure messages are only ACKed after successful processing. Dead Letter Topic (DLT) recovery after 3 retries with 1 second backoff between attempts.

- AckMode.MANUAL_IMMEDIATE — commit offset only after explicit ack.acknowledge() call
- DefaultErrorHandler — catches processing exceptions, retries 3 times with FixedBackOff(1000ms, 3)
- DeadLetterPublishingRecoverer — after retries exhausted, routes message to topic.DLT
- concurrency=3 — 3 consumer threads per listener, tune to match partition count
- AUTO_OFFSET_RESET=earliest — new consumer groups start from the beginning
- MAX_POLL_RECORDS=100 — processes up to 100 records per poll loop

4.3 Topic Configuration (KafkaTopicConfig.java)

Topics are declared as Spring @Bean instances of NewTopic. Spring Boot auto-creates them via Kafka Admin if they don't exist. All topics use 3 partitions and 3 replicas for production fault tolerance.

Topic Name	Retention / Purpose
healthcare.appointments.scheduled	7 days — new appointment bookings
healthcare.appointments.cancelled	7 days — cancellation events with reason
healthcare.patients.registered	7 days — new patient registrations
healthcare.billing.invoices	30 days — invoice generation events
healthcare.billing.payments	30 days — payment receipt events
healthcare.audit.events	90 days — all audit trail entries

4.4 Domain Event Producer (DomainEventProducer.java)

Publishes DomainEvent instances to the correct Kafka topic using Java 17 sealed switch expressions for exhaustive, compiler-enforced routing. The partition key is always the patientId so all events for the same patient land on the same partition — preserving event ordering per patient. Custom Kafka headers carry eventId, eventType, and source for downstream consumers and observability tools.

4.5 Consumers

AppointmentEventConsumer.java

Uses @RetryableTopic for non-blocking retries. Unlike traditional error handler retries which pause the partition, @RetryableTopic routes failed messages to a dedicated retry topic (healthcare.appointments.scheduled-retry) so the main partition continues processing. After all retries are exhausted, the message moves to the DLT (healthcare.appointments.scheduled-dlt). A separate @KafkaListener on the DLT logs failures for manual intervention.

BillingEventConsumer.java

Demonstrates batch consumption using batch=true on the @KafkaListener. Receives a List<ConsumerRecord> per poll and processes them together — useful for bulk Spanner inserts or analytics aggregation pipelines. Registers Micrometer counters for invoicesProcessed and paymentsProcessed that are exported to Prometheus.

5. Resilience — Circuit Breakers & Retries

5.1 Circuit Breaker Logic (ResilienceConfig.java)

Three named circuit breaker instances protect GCP service calls from cascading failures. Circuit breaker state transitions are logged as WARN for observability:

Instance	Protects	Opens at	Wait duration
spannerCB	Cloud Spanner queries/writes	50% failure rate	10 seconds in OPEN state
gcsCB	GCS upload/download/list	60% failure rate	15 seconds in OPEN state
secretMgrCB	Secret Manager reads	70% failure rate	30 seconds in OPEN state

State machine: CLOSED (normal) → OPEN (failing) → HALF_OPEN (probing) → CLOSED (recovered). In HALF_OPEN state, 5 test calls are permitted. If they succeed, the breaker closes. If they fail, it re-opens.

5.2 Retry Policies

- kafkaRetry — 3 attempts, 500ms fixed wait. Ignores IllegalArgumentException and IllegalStateException (not retryable — fix the data)
- spannerRetry — 4 attempts, exponential backoff starting at 200ms, doubling each attempt (200ms, 400ms, 800ms). Handles transient Spanner UNAVAILABLE errors

5.3 Resilient GCS Service (ResilientGcsService.java)

Demonstrates the Resilience4j Decorators functional builder pattern. Unlike @CircuitBreaker AOP annotations, this approach wraps Supplier<T> lambdas explicitly — making the resilience logic visible, testable without Spring context, and composable. Fallbacks return safe defaults (empty byte array, empty list) so callers continue functioning in degraded mode.

6. Security — Spring Security 6 + JWT

6.1 SecurityConfig.java

Configures a stateless OAuth2 JWT resource server using Spring Security 6's lambda DSL. No sessions, no CSRF (stateless API). JWT tokens are validated against GCP Identity Platform's JWKS endpoint at https://www.googleapis.com/oauth2/v3/certs.

Access Rules

Endpoint Pattern	Required Role	Description
/actuator/health/**, /actuator/info	Public	GKE liveness/readiness probes — no auth
/actuator/prometheus	Public	Prometheus scrape endpoint
GET /api/v1/**	Any authenticated	Read operations accessible to all users
POST/PATCH/DELETE /api/v1/appointments/**	CLINICIAN, ADMIN	Write operations require clinical role
/api/v1/billing/**	BILLING, ADMIN	Finance operations restricted to billing team
/api/pubsub/**, /api/storage/**	ADMIN	Infrastructure operations admin-only

JWT Role Extraction

The JwtGrantedAuthoritiesConverter reads the roles claim from the JWT payload and maps each value to a Spring ROLE_ authority. GCP Identity Platform encodes custom claims in the token using custom attributes or Firebase claims.

7. Monitoring & Observability

7.1 Custom Metrics (AppMetrics.java)

Registers business-level Micrometer metrics that go beyond default JVM/HTTP metrics:

Metric Name	Type	What it measures
app.appointments.scheduled.total	Counter	Total appointments ever scheduled
app.appointments.cancelled.total	Counter	Total appointments cancelled
app.billing.invoices.generated	Counter	Total invoices created
app.billing.payments.processed	Counter	Total payments successfully processed
app.kafka.publish.errors.total	Counter	Failed Kafka publish attempts
app.kafka.publish.duration.seconds	Timer (p50/p95/p99)	Kafka publish latency percentiles
app.appointments.active	Gauge	Current active (non-cancelled) appointments
app.kafka.consumer.lag	Gauge	Estimated consumer lag for scheduling topic
app.gcs.upload.bytes	Distribution Summary	Upload payload sizes in bytes (p50/p95)

AppMetrics also implements HealthIndicator — if kafkaPublishErrors exceeds 100, the custom health check at /actuator/health/appMetrics reports DOWN, triggering Kubernetes readiness probe failure.

7.2 GCP Health Indicator (GcpHealthIndicator.java)

A named HealthIndicator (gcpServices) that probes GCS and Pub/Sub availability. Exposed at /actuator/health/gcpServices. In Kubernetes, this can be used to delay traffic to the pod until GCP dependencies are confirmed healthy. Failed checks are logged as WARN with the error message for quick diagnosis.

7.3 Actuator Endpoints

- /actuator/health — overall health + sub-indicators (gcpServices, appMetrics, circuitBreakers, diskSpace, ping)
- /actuator/health/liveness — Kubernetes liveness probe — is the app alive?
- /actuator/health/readiness — Kubernetes readiness probe — is the app ready for traffic?
- /actuator/prometheus — Prometheus-format metrics scrape endpoint
- /actuator/metrics — JSON metrics browser
- /actuator/circuitbreakers — real-time circuit breaker state (CLOSED/OPEN/HALF_OPEN)

8. Batch Jobs & Scheduled Tasks

8.1 Audit Log Service (AuditLogService.java)

Implements an append-only compliance audit trail using a write-behind buffer pattern. The pattern works in three stages:

Stage 1 — Async Buffer: The record() method is annotated @Async so callers return immediately without waiting for I/O. Entries are added to a CopyOnWriteArrayList which is thread-safe for concurrent appends from multiple request threads.

Stage 2 — Scheduled Flush: Every 30 seconds (configurable via audit.flush.interval-ms), the flushToGcs() method drains the buffer using a copy-then-remove pattern. This avoids losing entries added while the flush is in progress.

Stage 3 — GCS Write: Entries are serialized to NDJSON (Newline Delimited JSON — one JSON object per line) and written to GCS under a date-partitioned path (audit/YYYY/MM/DD/timestamp.ndjson). This format is directly queryable by BigQuery external tables for analytics. If the GCS write fails, entries are re-added to the buffer so no audit records are lost.

AuditEntry is a Java 17 record: auditId, timestamp, action, entityType, entityId, actorId.

8.2 Overdue Invoice Job (OverdueInvoiceJob.java)

A @Scheduled cron job that runs daily at 02:00 UTC. Scans all PENDING invoices in Spanner, finds those whose dueDate is before the current time, updates their status to OVERDUE, and publishes a Pub/Sub notification to the notifications topic. The notification payload includes invoiceId, patientId, and dueDate as JSON. The job is idempotent — safe to re-run if it fails partway through, as OVERDUE invoices are already filtered out on the next run.

9. Angular 17 Frontend

9.1 Architecture Decisions

Angular 17 Features Used

Standalone components (no NgModule), functional guards, functional interceptors, Signals API for reactive state, @if/@for control flow syntax, inject() function

9.2 Application Config (app.config.ts)

Uses the standalone application config pattern introduced in Angular 17 — no AppModule class needed. Registers all providers using the new functional API: provideRouter(), provideHttpClient(), provideAnimations(). HTTP interceptors are registered as pure functions via withInterceptors() — no class-based interceptors.

9.3 Routing (app.routes.ts)

All feature modules are lazy-loaded using loadComponent() with dynamic imports. This means each feature's JavaScript bundle is only downloaded when the user navigates to that route — significantly reducing initial load time. Route guards protect access:

- authGuard — checks for access_token in localStorage. Redirects to /login if missing
- roleGuard — decodes the JWT payload, reads the roles claim, checks against route data.roles. Redirects to /dashboard if insufficient permissions

9.4 HTTP Interceptors (auth.interceptor.ts)

Three functional interceptors are chained via withInterceptors():

- authInterceptor — reads access_token from localStorage and attaches Authorization: Bearer <token> header to every outbound request
- correlationInterceptor — generates a UUID for each request and sets X-Correlation-Id header so the Spring Boot backend can trace it through MDC logs
- errorInterceptor — globally handles 401 (clears token and redirects to /login), 403 (logs forbidden), and network errors (status 0)

9.5 Angular Signals — Dashboard (dashboard.component.ts)

The dashboard uses Angular 17 Signals instead of BehaviorSubject for local component state. signal() creates a reactive value, computed() derives values from signals automatically, and the template re-renders only when signal values change. This is more efficient than zone.js-based change detection for fine-grained updates.

```
stats   = signal<DashboardStats | null>(null);
loading = signal(true);
statCards = computed(() => { /* derives from stats() */ });
```

9.6 Appointment List (appointment-list.component.ts)

Uses Angular Material's mat-table with matSort for sortable columns. Client-side filtering is implemented reactively using FormControl.valueChanges piped through debounceTime(300ms) and distinctUntilChanged — avoids

filtering on every keystroke. Status chips are color-coded using CSS classes mapped to AppointmentStatus values.

9.7 Appointment Form (appointment-form.component.ts)

Demonstrates Angular Reactive Forms with FormBuilder. Each form control has validators (Validators.required). The mat-datepicker provides a calendar UI for selecting the appointment date. On submit, the date is converted to ISO string before sending to the REST API. Error states are displayed using the new @if control flow syntax.

9.8 Services

All Angular services use inject() instead of constructor injection (Angular 17 best practice). AppointmentService maintains a BehaviorSubject<Appointment[]> as a local cache that components can subscribe to — avoiding redundant HTTP calls. PatientService and BillingService follow standard HttpClient patterns with RxJS map() operators to unwrap the ApiResponse<T> wrapper.

10. React 18 Frontend

10.1 Tech Stack

Library	Role
Vite	Build tool — faster than Create React App, native ESM, HMR under 50ms
React 18	UI framework — concurrent features, automatic batching, Suspense
Redux Toolkit	Global state — appointments, patients, auth slice with createSlice
React Query	Server state — caching, background refetch, optimistic updates
React Hook Form	Form state — minimal re-renders, Zod schema validation
Zod	Schema validation — type-safe form validation with TypeScript inference
Tailwind CSS	Utility-first styling — responsive design without custom CSS
Recharts	Charts — appointment trends, billing revenue dashboards
Axios	HTTP client — interceptors for JWT injection and error handling
Lucide React	Icon library — consistent SVG icons across all components

10.2 TypeScript Domain Types (src/types/index.ts)

All domain types mirror the Spring Boot DTOs exactly, ensuring the frontend and backend stay in sync. Types cover Patient, Appointment, Invoice, DomainEvent variants, and ApiResponse<T> generic wrapper. PatientStatus, AppointmentStatus, and InvoiceStatus are TypeScript union types providing exhaustive type checking in switch statements — the same safety guarantee as Java sealed classes on the backend.

11. Infrastructure & DevOps

11.1 Dockerfile (multi-stage build)

Uses a two-stage Docker build to minimize the final image size:

- Stage 1 (builder): Uses eclipse-temurin:17-jdk-alpine with Maven to compile and package the JAR. The full JDK and Maven are only in this intermediate layer.
- Stage 2 (runtime): Copies only the JAR from Stage 1 into eclipse-temurin:17-jre-alpine. The JRE is significantly smaller than the JDK. A non-root user (appuser) is created and the container runs as this user for security.

JVM flags: -XX:+UseContainerSupport enables container-aware memory limits. -XX:MaxRAMPercentage=75.0 allows the JVM to use 75% of the container's memory limit. -Djava.security.egd=file:/dev/./urandom speeds up random number generation for faster startup.

11.2 Kubernetes Manifests (k8s/)

configmap.yml

Stores all non-sensitive configuration as a ConfigMap: GCP project ID, region, Spanner instance/database names, GCS bucket name, and Pub/Sub topic/subscription names. The ConfigMap is mounted as envFrom in the Deployment so all values become environment variables in the container.

deployment.yml

Configures a rolling update deployment with 2 replicas and zero-downtime settings (maxUnavailable: 0, maxSurge: 1). Resource requests (250m CPU, 512Mi RAM) and limits (1000m CPU, 1Gi RAM) are set for predictable scheduling by the Kubernetes scheduler. Health probes point to /actuator/health/liveness and /actuator/health/readiness respectively. The GCP service account key is mounted as a read-only secret volume at /var/secrets/google.

service.yml

Creates a LoadBalancer Service that exposes port 80 externally, forwarding to container port 8080. GKE automatically provisions a Google Cloud Load Balancer with an external IP address.

11.3 GitHub Actions Pipeline (.github/workflows/gcp-deploy.yml)

A three-job sequential pipeline triggered on every push to main that touches gcp-springboot/**:

Job	Runs on	Steps
build-and-test	ubuntu-latest	Checkout → Java 17 setup with Maven cache → mvn test → mvn package -DskipTests → upload JAR artifact
docker-push	ubuntu-latest	Authenticate to GCP (sa key) → configure Docker for GCR → download JAR → docker build (tagged with SHA + latest) → docker push
deploy-to-gke	ubuntu-latest	Authenticate to GCP → get GKE credentials → sed replace IMAGE_PLACEHOLDER → kubectl apply -f

Job	Runs on	Steps
		k8s/ → kubectl rollout status (120s timeout) → print service IP

Required GitHub Secrets: GCP_PROJECT_ID, GCP_SA_KEY, GCP_REGION, GKE_CLUSTER_NAME, SPANNER_INSTANCE_ID, SPANNER_DATABASE, GCS_BUCKET_NAME.

12. Testing Strategy

12.1 Unit Tests

Test Class	What it verifies
AppointmentServiceTest	Schedule creates appointment + publishes AppointmentScheduled event. Cancel publishes AppointmentCancelled. Confirm publishes no event. Missing ID returns empty Optional
DomainEventProducerTest	Parameterized test verifies each of 5 event types routes to the correct Kafka topic via sealed switch. Separate test verifies patientId is used as partition key
GcsStorageServiceTest	Download returns bytes when blob exists, empty array when not found. Delete returns true/false based on GCS response
PubSubPublisherServiceTest	Publish resolves with message ID on success. Attributes are passed through. CompletableFuture completes exceptionally on template failure

12.2 Integration Tests

KafkaIntegrationTest.java

Uses @EmbeddedKafka to start a real in-process Kafka broker with all 5 topic names pre-created. Publishes an AppointmentScheduled event via DomainEventProducer and asserts it arrives on the correct topic within 5 seconds using a BlockingQueue test listener. Verifies the topic name, partition key (patientId), and event payload are all correct. Uses @SpringBootTest so the full application context is loaded — including the real KafkaTemplate with JSON serialization.

12.3 Controller Slice Tests

AppointmentControllerTest.java

Uses @WebMvcTest which loads only the web layer (controllers, filters, security config) without Kafka or GCP. @MockBean replaces AppointmentService with a Mockito mock. MockMvc performs HTTP requests and asserts response status codes, JSON body structure, and security:

- POST /api/v1/appointments with CLINICIAN role returns 201 with appointment data
- GET /api/v1/appointments/{id} returns 200 when found, 404 when not
- POST with missing patientId returns 400 (Bean Validation)
- POST without authentication returns 401 (Spring Security)

13. Configuration (application.yml)

The application uses a single application.yml with environment variable substitution for all environment-specific values. Default values are provided for local development. In production (GKE), environment variables override the defaults via the Kubernetes ConfigMap and Secrets.

Config Property	Env Variable	Description
spring.cloud.gcp.project-id	GCP_PROJECT_ID	GCP project ID for all GCP SDK clients
spring.cloud.gcp.credentials.location	GOOGLE_APPLICATION_CREDENTIALS	Path to service account JSON key
spring.cloud.gcp.spanner.instance-id	SPANNER_INSTANCE_ID	Spanner instance name
spring.cloud.gcp.spanner.database	SPANNER_DATABASE	Spanner database name
gcs.bucket-name	GCS_BUCKET_NAME	GCS bucket for file storage
spring.kafka.bootstrap-servers	KAFKA_BOOTSTRAP_SERVERS	Kafka broker addresses (comma-separated)
spring.security.oauth2.resourceserver.jwt.jwk-set-uri	JWT_JWK_SET_URI	JWKS endpoint for JWT validation

14. Complete Project Structure

```
Dev/
├── gcp-springboot/
│   ├── Dockerfile
│   ├── pom.xml
│   ├── k8s/
│   │   ├── configmap.yml
│   │   ├── deployment.yml
│   │   └── service.yml
│   └── src/main/java/com/navya/gcp/
│       ├── GcpApplication.java
│       ├── batch/
│       │   ├── AuditLogService.java
│       │   └── OverdueInvoiceJob.java
│       ├── config/
│       │   └── GlobalExceptionHandler.java
│       ├── domain/
│       │   ├── dto/Dtos.java
│       │   ├── event/DomainEvent.java
│       │   └── model/ (Appointment.java, Billing.java)
│       ├── kafka/
│       │   ├── config/ (Producer, Consumer, Topic configs)
│       │   ├── consumer/ (Appointment, Billing consumers)
│       │   └── producer/ DomainEventProducer.java
│       ├── monitoring/ (AppMetrics, GcpHealthIndicator)
│       ├── pubsub/ (Publisher, Subscriber, Handler, Controller)
│       ├── resilience/ (ResilienceConfig, ResilientGcsService)
│       ├── rest/
│       │   ├── appointment/ (Controller, Service, Repository)
│       │   ├── billing/ (Controller, Service, Repository)
│       │   └── filter/ (CorrelationId, RequestLogging)
│       ├── secretmanager/ (Service, Config)
│       ├── security/ SecurityConfig.java
│       ├── spanner/ (Patient entity, Repository, Service, Controller)
│       └── storage/ (GcsStorageService, GcsController)
│
├── angular-ui/
│   └── src/app/
│       ├── app.config.ts, app.routes.ts
│       ├── core/
│       │   ├── guards/ (auth.guard.ts, login.component.ts)
│       │   ├── interceptors/ (auth.interceptor.ts)
│       │   ├── models/ models.ts
│       │   └── services/ (appointment.service.ts, api.services.ts)
│       └── features/
│           ├── dashboard/ DashboardComponent
│           ├── appointments/ (ListComponent, FormComponent)
│           ├── patients/ (ListComponent, DetailComponent)
│           └── billing/ InvoiceListComponent
```

```
│
├── react-ui/
│   ├── package.json  (Vite + React 18 + Redux + RQ + Tailwind)
│   └── src/types/index.ts
│
└── .github/workflows/gcp-deploy.yml
```

15. Quick Reference — Interview Topics

Java 17 Features Used

- Sealed interfaces — DomainEvent hierarchy with compiler-enforced exhaustiveness
- Records — all DTOs, AuditEntry, event payload types (immutable, no boilerplate)
- Pattern matching switch — DomainEventProducer topic routing and key selection
- var inference — used in service methods for local variable declarations
- Text blocks — multi-line SQL strings in SpannerTemplate queries

Spring Boot 3 Features Used

- @SpringBootApplication — auto-configuration, component scan, bean registration
- @RestControllerAdvice + ProblemDetail — RFC 7807 structured error responses
- @Transactional — Spanner write operations with rollback on exception
- @Scheduled — cron-based overdue invoice detection (0 0 2 * * * UTC)
- @Async — non-blocking audit log buffering
- HttpInterceptorFn — Angular 17 functional HTTP interceptors
- spring.config.import: sm:// — Secret Manager property source

Kafka Concepts Demonstrated

- Idempotent producer — prevents duplicate messages on retry
- Manual offset commit — ACK only after successful processing
- Dead Letter Topic — @RetryableTopic with configurable retry attempts and backoff
- Partition key strategy — patientId as key ensures ordered events per patient
- Batch consumer — List<ConsumerRecord> per poll for bulk processing
- Topic provisioning — NewTopic beans with partition/replica/retention config

GCP Services Demonstrated

- Pub/Sub — publisher with attributes, push subscriber with ACK/NACK, emulator support
- Cloud Spanner — Spring Data repository, @Transactional, SpannerTemplate for raw SQL
- Secret Manager — CRUD SDK, sm:// Spring property injection
- GCS — upload, download, stream, signed URLs, list with prefix, delete

Document generated by Claude · Healthcare Platform · github.com/nakurathi/Dev